

# 1. Introduction

---

## Objectif de l'API

L'API développée pour ce projet a pour objectif de servir de **pont entre le site internet et la base de données**, en permettant aux différentes interfaces (frontend, tableau d'administration, formulaires) d'interagir avec les données de manière fluide, sécurisée et structurée.

## Finalités pédagogiques

Cette API est conçue pour des étudiants en développement web afin de :

- Comprendre le fonctionnement d'une **API RESTful** dans un contexte concret.
- Apprendre à **exposer et consommer des données** via des requêtes HTTP (GET, POST, PUT, DELETE).
- Gérer des ressources courantes d'un site internet (ex. pages, utilisateurs, messages de contact, articles, etc.).
- Mettre en place des **mécanismes d'authentification** et de **rôles** (ex. admin vs visiteur).
- Organiser les routes et les contrôleurs de façon modulaire et maintenable.

## Objectifs fonctionnels

L'API permet notamment :

- L'affichage dynamique du contenu du site (pages, articles, sections, témoignages...).
- La gestion des utilisateurs (authentification, inscription, profils).
- L'enregistrement de données issues des formulaires (ex. formulaire de contact).
- L'administration des contenus via une interface back-office connectée à l'API.
- Le respect des standards REST pour garantir une bonne interopérabilité et faciliter le développement côté frontend.

## Cas d'usage typiques :

- Un utilisateur visite la page d'accueil : le frontend envoie une requête GET `/pages/home` à l'API.
- Un administrateur ajoute un article via le CMS : une requête POST `/articles` est envoyée avec le contenu.
- Un visiteur envoie un message via le formulaire de contact : une requête POST `/messages` est déclenchée.

- Un utilisateur se connecte à son espace personnel : une requête **POST /auth/login** est envoyée pour obtenir un token d'accès.

## Technologies utilisées

L'API repose sur un ensemble de technologies web modernes, choisies pour leur simplicité, leur large adoption dans l'industrie, et leur pertinence pédagogique. Elles permettent de créer une architecture robuste, sécurisée et facile à maintenir.



### Backend

- **Node.js** : Environnement d'exécution JavaScript côté serveur.
- **Express.js** : Framework minimaliste et flexible pour créer des routes, middlewares et gérer les requêtes HTTP.
- **MongoDB / PostgreSQL / MySQL** (*au choix selon le projet*) : Base de données pour stocker les utilisateurs, contenus, messages, etc.
- **Mongoose / Sequelize / Prisma** (*selon la BDD*) : ORM/ODM permettant de manipuler les données plus facilement.



### Sécurité & Authentification

- **JWT (JSON Web Tokens)** : Système de tokens pour gérer l'authentification stateless (sans session serveur).
- **Bcrypt** : Pour le hachage sécurisé des mots de passe.
- **CORS** : Middleware pour gérer les autorisations entre domaines (ex. frontend et backend séparés).



### Outils de développement

- **Nodemon** : Redémarrage automatique du serveur pendant le développement.
- **Postman / Insomnia** : Tests manuels de l'API.
- **dotenv** : Gestion des variables d'environnement (ex. clés secrètes, URL de BDD).



### Documentation & Validation

- **Swagger / Postman Docs** (*optionnel*) : Pour générer une documentation interactive de l'API.
- **Joi / express-validator** : Pour valider les données reçues dans les requêtes (formulaires, etc.).

## Frontend (interactions avec l'API)

Bien que cette section se concentre sur l'API, celle-ci est conçue pour interagir avec :

- **HTML/CSS/JS Vanilla** ou
  - **Frameworks modernes** : React, Vue.js, ou même un CMS headless comme Next.js avec API externe.
- 

Voici le développement de la section "**Accès à l'API**", essentiel pour que les étudiants comprennent **où** et **comment** consommer l'API :

---

## Accès à l'API

L'API est accessible via une **URL de base** (base URL) à partir de laquelle toutes les requêtes sont construites. Cette base est commune à tous les endpoints, ce qui permet une structure claire et cohérente.

### URL de base

`https://api.monsite.com/v1`

Remarque : En local, l'API peut être accessible via :

`http://localhost:3000/api/v1`

Le segment `/v1` correspond à la **version** de l'API, permettant d'assurer une compatibilité future si l'API évolue (ex. `/v2`, `/v3`, etc.).

### Exemple d'appel d'API

Pour récupérer la liste des articles de blog :

GET `https://api.monsite.com/v1/articles`

Pour se connecter :

POST `https://api.monsite.com/v1/auth/login`

### Headers courants à inclure dans les requêtes

Content-Type: `application/json`

Authorization: Bearer `<votre_token>`

- **Content-Type** : précise que les données envoyées ou reçues sont en JSON.
- **Authorization** : utilisé pour les routes protégées ; nécessite un **token JWT** obtenu lors de la connexion.

## Accès en local (pendant le développement)

Quand vous développez votre API en local :

- Backend lancé sur `localhost:3000`
- Requêtes vers : `http://localhost:3000/api/v1/...`

Vous pouvez utiliser **Postman**, **Insomnia** ou **fetch** / **axios** dans le frontend pour tester ces routes.



## 2. Authentification

---

L'authentification permet de sécuriser certaines routes de l'API (comme la gestion de contenu ou l'accès à des profils utilisateurs) en s'assurant que seul un utilisateur autorisé peut y accéder. L'API utilise un système de **tokens** basé sur les normes **JWT** et **Bearer**.



### Types de token utilisés



#### JWT (JSON Web Token)

Un **JWT** est un jeton sécurisé encodé en base64 qui contient :

- des informations (payload) comme l'id de l'utilisateur, son rôle, etc.
- une signature cryptographique pour garantir son authenticité

#### Exemple de JWT :

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEyMyIsInJvbGU0IjoiJhZG1pbiIsImVudCI6MTY1OTg0NzA0NH0.4c9gsvE_oYzqY2Wp3...
```

Ce token est généralement généré au moment de la **connexion** (POST /auth/login) ou de l'**inscription** (POST /auth/register).



#### Bearer Token

Le token JWT est envoyé dans le **header Authorization** sous le format suivant :

Authorization: Bearer <votre\_token>

#### Exemple :

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...

Le mot "**Bearer**" signifie que le client *porte* (bear) le token pour prouver son identité à chaque requête.



### Cycle d'authentification simplifié

1. L'utilisateur envoie ses identifiants via :

POST /auth/login

2. Le serveur vérifie les informations et retourne un **JWT** :

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

3. Le frontend stocke ce token (souvent en localStorage ou sessionStorage).
4. À chaque requête vers une route protégée, le frontend ajoute ce token dans les **headers**.

### Exemple d'utilisation (avec fetch)

```
fetch("https://api.monsite.com/v1/articles", {  
  method: "GET",  
  headers: {  
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs...",  
    "Content-Type": "application/json"  
  }  
});
```

## Endpoints de Connexion / Inscription

Ces endpoints permettent aux utilisateurs de **créer un compte**, de **se connecter** pour obtenir un token d'authentification (JWT), et, le cas échéant, de **se déconnecter** ou vérifier leur session.

### ♦ **POST /auth/register – Inscription**

Permet à un nouvel utilisateur de créer un compte.

#### Requête

POST /auth/register  
Content-Type: application/json

```
{  
  "username": "johndoe",  
  "email": "johndoe@example.com",  
  "password": "MotDePasse123"  
}
```

#### Réponse

```
{  
  "message": "Utilisateur inscrit avec succès",  
  "user": {  
    "id": "123456",  
    "username": "johndoe",  
    "email": "johndoe@example.com"  
  },  
  "token": "eyJhbGciOiJIUzI1NiIsInR..."  
}
```

Le token peut être stocké côté client pour authentifier les futures requêtes.

### ♦ **POST /auth/login – Connexion**

Permet à un utilisateur existant de se connecter et d'obtenir un token JWT.



#### **Requête**

POST /auth/login

Content-Type: application/json

```
{
  "email": "johndoe@example.com",
  "password": "MotDePasse123"
}
```



#### **Réponse**

```
{
  "message": "Connexion réussie",
  "user": {
    "id": "123456",
    "username": "johndoe"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5In0..."
}
```

Ce token devra ensuite être envoyé dans le header de chaque requête protégée :

Authorization: Bearer <token>

### ♦ **POST /auth/logout (optionnel)**

Dans un système stateless (comme avec JWT), la déconnexion est souvent **gérée côté frontend** en supprimant le token stocké.

Mais on peut tout de même prévoir un endpoint pour **invalider le token** côté serveur, si besoin.



#### **Routes protégées**

Une fois connecté, les routes suivantes sont généralement protégées par un middleware :

- GET /users/me
- PUT /users/:id
- POST /articles
- etc.



## Exemple de Headers d'Autorisation

Pour accéder aux **routes protégées** de l'API (comme récupérer les infos de son profil, publier un article, etc.), l'utilisateur doit envoyer un **token JWT** dans le header HTTP de sa requête. Ce token lui est fourni lors de la **connexion**.



### Format attendu

Authorization: Bearer <votre\_token>

- `Authorization` : nom du header.
- `Bearer` : mot-clé qui précède le token (standard).
- `<votre_token>` : le **token JWT** retourné par le serveur au moment de la connexion.



### Exemple complet (avec fetch)

```
fetch("https://api.monsite.com/v1/users/me", {
  method: "GET",
  headers: {
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "Content-Type": "application/json"
  }
});
```



### Exemple dans Postman

1. Va dans l'onglet **Headers**.
2. Ajoute une ligne :

| Key | Value |
|-----|-------|
|-----|-------|

|               |                                                |
|---------------|------------------------------------------------|
| Authorization | Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... |
|---------------|------------------------------------------------|

Tu peux aussi utiliser l'onglet **Authorization** > Type : *Bearer Token* dans Postman, ce qui le remplira automatiquement.



### Sans token valide

Si le token est manquant, expiré ou invalide, le serveur renverra une erreur :

```
{
  "error": "Token manquant ou invalide"
}
```

Code de réponse : **401 Unauthorized** ou **403 Forbidden** selon la logique de l'API.

---



## 3. Structure des Requêtes

---

Les requêtes envoyées à l'API doivent respecter une certaine structure afin d'être bien comprises et traitées par le serveur. Voici les points essentiels à connaître.

---

### Format des requêtes

#### JSON (format principal utilisé)

Les données envoyées dans le corps (body) des requêtes POST, PUT ou PATCH sont généralement encodées en **JSON**.

Content-Type: application/json

**Exemple** – Création d'un article :

```
{
  "title": "Bienvenue sur notre blog",
  "content": "Voici notre premier article",
  "authorId": "12345"
}
```

#### URL-encoded (plus rare)

Utilisé parfois pour des formulaires HTML simples, mais déconseillé pour une API moderne.

Content-Type: application/x-www-form-urlencoded

### Headers requis

Pour la majorité des requêtes, les headers suivants sont recommandés :

| Header        | Description                                                   |
|---------------|---------------------------------------------------------------|
| Content-Type  | Définit le format des données envoyées (application/json)     |
| Authorization | Obligatoire pour les routes protégées. Contient le token JWT. |

**Exemple :**

Content-Type: application/json  
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

### Pagination, tri, filtres (si applicable)

Pour les ressources listées (articles, produits, utilisateurs, etc.), l'API peut proposer des **paramètres de requête (query params)** pour gérer la pagination, le tri et les filtres.



## Pagination

GET /articles?page=2&limit=10

- `page` : numéro de page
- `limit` : nombre d'éléments par page



## Tri

GET /articles?sort=createdAt&order=desc

- `sort` : champ de tri (ex. `createdAt`, `title`, etc.)
- `order` : `asc` (croissant) ou `desc` (décroissant)



## Filtres

GET /articles?author=johndoe&category=tech

- Filtres personnalisés selon les champs disponibles sur la ressource.



## Exemple complet

GET /articles?category=tech&page=1&limit=5&sort=createdAt&order=desc

**Résultat attendu** : les 5 premiers articles de la catégorie "tech", triés du plus récent au plus ancien.



## 4. Réponses de l'API

---

Toutes les réponses de l'API sont retournées au format **JSON**. Cela permet une communication claire et standardisée entre le serveur et les clients (ex : applications front-end, outils comme Postman, etc.).

---



### Format de réponse (JSON)

Une réponse réussie contient généralement :

- Un **code de statut HTTP** approprié (200, 201, etc.)
- Un **message**
- Les **données** retournées (si applicable)



#### Exemple – Succès 200 OK

GET /articles/123

```
{
  "message": "Article trouvé",
  "data": {
    "id": "123",
    "title": "Bienvenue sur notre blog",
    "content": "Voici notre premier article",
    "author": "admin"
  }
}
```



#### Exemple – Création réussie 201 Created

POST /articles

```
{
  "message": "Article créé avec succès",
  "data": {
    "id": "456",
    "title": "Nouvel article",
    "content": "Contenu ici..."
  }
}
```

## ❌ Codes d'erreur et gestion des statuts HTTP

Voici les codes d'état les plus courants, avec leur signification :

| Code | Signification         | Quand l'utiliser ?                 |
|------|-----------------------|------------------------------------|
| 200  | OK                    | Requête réussie (GET, PUT)         |
| 201  | Created               | Ressource créée avec succès (POST) |
| 400  | Bad Request           | Données invalides ou manquantes    |
| 401  | Unauthorized          | Token manquant ou invalide         |
| 403  | Forbidden             | Accès refusé (droits insuffisants) |
| 404  | Not Found             | Ressource inexistante              |
| 500  | Internal Server Error | Erreur côté serveur                |

### ⚠️ Exemple de messages d'erreur

#### ❌ Requête invalide – 400 Bad Request

```
{  
  "error": "Le champ 'email' est requis"  
}
```

#### 🔒 Non autorisé – 401 Unauthorized

```
{  
  "error": "Token d'authentification manquant ou invalide"  
}
```

#### 🚫 Accès interdit – 403 Forbidden

```
{  
  "error": "Vous n'avez pas les droits pour effectuer cette action"  
}
```

#### ? Ressource non trouvée – 404 Not Found

```
{  
  "error": "Article non trouvé"  
}
```

#### 💥 Erreur serveur – 500 Internal Server Error

```
{  
  "error": "Une erreur est survenue, veuillez réessayer plus tard"  
}
```

✅ **Bonnes pratiques** : toujours afficher clairement les erreurs côté client en fonction du code HTTP et du message retourné par l'API.

## 5. Endpoints par Ressource

---

Cette section décrit les **routes principales** de l'API, regroupées par ressource (utilisateurs, authentification, etc.). Chaque endpoint inclut la méthode HTTP utilisée, la structure attendue, et le comportement de l'API.

---

### ◆ Utilisateurs

#### **GET /users – Lister les utilisateurs**

-  Protégé (admin uniquement)
- Renvoie une liste de tous les utilisateurs

```
GET /users
Authorization: Bearer <admin_token>
```

#### **GET /users/:id – Voir un utilisateur**

- Accessible à l'utilisateur concerné ou à un admin

```
GET /users/123
Authorization: Bearer <token>
```

#### **POST /users – Créer un utilisateur (via back-office ou API ouverte)**

- Identique à /auth/register mais côté admin

```
POST /users
{
  "username": "paul",
  "email": "paul@example.com",
  "password": "secret123"
}
```

#### **PUT /users/:id – Modifier un utilisateur**

- Permet de modifier son profil ou celui d'un autre (si admin)

```
PUT /users/123
{
  "username": "paul_updated"
}
```

#### **DELETE /users/:id – Supprimer un utilisateur**

- Supprime le compte utilisateur (admin ou utilisateur lui-même)

```
DELETE /users/123
```

## ◆ Authentication

### **POST /auth/register – Inscription**

Crée un compte utilisateur et renvoie un token.

```
POST /auth/register
{
  "email": "lucie@mail.com",
  "username": "lucie",
  "password": "motdepasse"
}
```

### **POST /auth/login – Connexion**

Authentifie un utilisateur et renvoie un token JWT.

```
POST /auth/login
{
  "email": "lucie@mail.com",
  "password": "motdepasse"
}
```

### **POST /auth/logout (*optionnel/stateless*)**

Peut servir à invalider un token (si blacklist côté serveur).

## ◆ Articles (exemple de ressource additionnelle)

### **GET /articles – Lister les articles**

```
GET /articles
```

### **GET /articles/:id – Voir un article**

```
GET /articles/123
```

### **POST /articles – Créer un article**

```
POST /articles
Authorization: Bearer <token>
{
  "title": "Mon premier article",
  "content": "Contenu de l'article"
}
```

### **PUT /articles/:id – Modifier un article**

```
PUT /articles/123
Authorization: Bearer <token>
{
  "title": "Article mis à jour"
}
```

### **DELETE /articles/:id – Supprimer un article**

```
DELETE /articles/123
Authorization: Bearer <token>
```

💡 **Remarque** : d'autres ressources peuvent être ajoutées selon le site : produits, commandes, messages, services, catégories, etc.

Souhaites-tu que je développe une autre ressource ou que je continue vers la section suivante : **6. Exemples de requêtes ?**



## 6. Exemples de Requêtes

---

Voici des exemples de requêtes pour consommer l'API depuis différents outils ou bibliothèques. Cela permet aux étudiants de tester l'API depuis le **navigateur**, des **scripts JavaScript**, ou des outils comme **Postman** ou **curl**.

---

### ◆ Requêtes avec **fetch** (JavaScript – navigateur ou Node.js)

#### Exemple : Connexion (POST /auth/login)

```
fetch("https://api.monsite.com/v1/auth/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    email: "test@mail.com",
    password: "monMotDePasse"
  })
})
.then(res => res.json())
.then(data => console.log(data));
```

#### Exemple : Obtenir la liste des articles (route protégée)

```
fetch("https://api.monsite.com/v1/articles", {
  headers: {
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."
  }
})
.then(res => res.json())
.then(data => console.log(data));
```

---

### ◆ Requêtes avec **axios** (JavaScript – plus haut niveau que fetch)

```
import axios from "axios";

axios.post("https://api.monsite.com/v1/auth/login", {
  email: "test@mail.com",
  password: "monMotDePasse"
})
.then(response => {
  const token = response.data.token;
  return axios.get("https://api.monsite.com/v1/articles", {
    headers: {
      Authorization: `Bearer ${token}`
    }
  });
})
.then(res => console.log(res.data));
```



### ♦ Requetes dans Postman

1. Sélectionner la méthode (GET, POST, etc.).
2. Entrer l'URL, ex : `https://api.monsite.com/v1/articles`
3. Onglet **Headers** :
  - Content-Type: `application/json`
  - Authorization: `Bearer <votre_token>`
4. Onglet **Body** (en raw > JSON) pour POST/PUT :

```
{  
  "title": "Nouveau titre",  
  "content": "Contenu de l'article"  
}
```

### ♦ Requetes avec curl (terminal / ligne de commande)

#### Connexion

```
curl -X POST https://api.monsite.com/v1/auth/login \  
  -H "Content-Type: application/json" \  
  -d '{"email":"test@mail.com","password":"monMotDePasse"}'
```

#### Récupérer les articles (avec token)

```
curl https://api.monsite.com/v1/articles \  
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."
```



## 7. Bonnes Pratiques

Respecter les bonnes pratiques lors du développement et de la consommation d'une API est essentiel pour garantir la **sécurité**, la **maintenabilité** et la **compatibilité** dans le temps.

---



### Sécurité



#### Protection des routes

- Les **routes sensibles** (modification, suppression, création) doivent être **protégées** par un middleware d'authentification.
- Exemple : pour accéder à `/users/me` ou créer un article, l'utilisateur doit être connecté et envoyer un **token JWT valide** dans le header.



#### Validation des données

- Toujours **valider les entrées** côté serveur avec une bibliothèque (comme `express-validator`, `Joi`, `Zod...`).
- Exemples de validation :
  - format de l'email
  - longueur minimale du mot de passe
  - champs obligatoires présents

Cela protège contre les **injections**, les **requêtes malformées**, et améliore la robustesse de l'API.



#### Sécurité des mots de passe

- Les mots de passe doivent **toujours être hashés** avec une bibliothèque comme `bcrypt` avant d'être stockés en base de données.



### Respect des conventions REST



#### Verbes HTTP

- GET : lire une ressource
- POST : créer une ressource
- PUT : mettre à jour complètement
- PATCH : mettre à jour partiellement
- DELETE : supprimer



## Structure des routes

- Utiliser des **noms de ressources au pluriel** :
  -  /users, /articles, /products
  -  /getUser, /createArticle



## Réponses claires

- Toujours retourner un **code HTTP adapté** (200, 201, 400, 401, 404, etc.)
  - Inclure des **messages explicites** et des **données utiles** dans la réponse JSON
- 



## Versionnage de l'API

Versionner une API permet de :

- gérer les évolutions sans casser les clients existants,
- tester de nouvelles fonctionnalités en parallèle,
- maintenir la rétrocompatibilité.



## Exemple :

`https://api.monsite.com/v1/users`

Si une nouvelle version est introduite :

`https://api.monsite.com/v2/users`

Le versionnage se fait généralement via l'URL (/v1/) ou dans les headers (Accept : `application/vnd.monsite.v1+json`).

---

Très bien ! Voici la section **8. Annexes**, qui sert de référence rapide et pédagogique pour les étudiants. Elle regroupe des notions importantes, des ressources utiles et des exemples concrets de **modèles de données**.

## 8. Annexes

---

### Glossaire

| Terme             | Définition                                                                                          |
|-------------------|-----------------------------------------------------------------------------------------------------|
| <b>API</b>        | Interface qui permet à une application d'accéder à des fonctionnalités d'un serveur ou d'un service |
| <b>REST</b>       | Style d'architecture pour construire des API web en utilisant les méthodes HTTP standards           |
| <b>Endpoint</b>   | URL spécifique permettant d'accéder à une ressource (ex: <code>/users/1</code> )                    |
| <b>Token JWT</b>  | Jeton signé contenant des infos utilisateur pour les requêtes sécurisées                            |
| <b>CRUD</b>       | Create, Read, Update, Delete – les 4 actions principales sur les données                            |
| <b>Middleware</b> | Fonction intermédiaire exécutée entre la requête et la réponse du serveur                           |
| <b>Header</b>     | Informations envoyées avec une requête ou une réponse HTTP                                          |
| <b>Payload</b>    | Contenu (souvent en JSON) envoyé dans le corps d'une requête POST/PUT                               |

---

### Liens utiles

- [MDN Web Docs – HTTP](#)
  - [JWT.io – Visualiser et comprendre un token JWT](#)
  - [RESTful API Design – Best practices](#)
  - [Postman – Tester des APIs facilement](#)
  - [JSON Formatter & Validator](#)
- 

### Modèles de données (schemas JSON)

#### Utilisateur

```
{
  "id": "u123",
  "username": "lucie",
  "email": "lucie@mail.com",
  "role": "user",
  "createdAt": "2024-01-15T10:20:00Z"
}
```

#### Article

```
{
  "id": "a456",
  "title": "Introduction à l'API",
  "content": "Cet article vous explique les bases d'une API REST...",
  "authorId": "u123",
  "createdAt": "2024-02-10T14:35:00Z",
  "tags": ["API", "REST"]
}
```

}

### **Message de contact (ex: formulaire de site)**

```
{  
  "id": "m789",  
  "name": "Jean Dupont",  
  "email": "jean.dupont@mail.com",  
  "message": "Bonjour, j'aimerais plus d'informations...",  
  "sentAt": "2024-06-18T09:10:00Z"  
}
```



Cette section peut être enrichie au fur et à mesure avec :

- des schémas UML ou MCD (modèle conceptuel de données),
- des exemples de relations entre les entités,
- ou des réponses d'API formatées réelles.

# **Projet API de blog — Étapes pas à pas (niveau débutant)**

---

## **Étape 1 : Initialiser le projet**

### 1. Créer un dossier de projet :

```
mkdir api-blog && cd api-blog
```

### 2. Initialiser Node.js :

```
npm init -y
```

### 3. Installer les dépendances :

```
npm install express cors body-parser
```

### 4. Créer l'arborescence :

```
api-blog/  
├── server.js  
├── data/  
│   └── articles.json  
└── routes/  
    └── articles.js
```

---

## **Étape 2 : Créer un serveur Express de base**

### **server.js**

```
const express = require("express");  
const cors = require("cors");  
const app = express();  
  
app.use(cors());  
app.use(express.json());  
  
// Routes  
const articleRoutes = require("./routes/articles");  
app.use("/api/articles", articleRoutes);  
  
const PORT = 3000;  
app.listen(PORT, () => {  
  console.log(`Serveur démarré sur http://localhost:${PORT}`);  
});
```

### Étape 3 : Créer les données (simples, en JSON)

**data/articles.json**

```
[
  {
    "id": 1,
    "title": "Bienvenue sur le blog",
    "content": "Voici le premier article."
  }
]
```

---

### Étape 4 : Créer les routes CRUD des articles

**routes/articles.js**

```
const express = require("express");
const fs = require("fs");
const router = express.Router();
const filePath = "../data/articles.json";

// Lister les articles
router.get("/", (req, res) => {
  const articles = JSON.parse(fs.readFileSync(filePath));
  res.json(articles);
});

// Voir un article
router.get("/:id", (req, res) => {
  const articles = JSON.parse(fs.readFileSync(filePath));
  const article = articles.find(a => a.id == req.params.id);
  if (article) res.json(article);
  else res.status(404).json({ error: "Article non trouvé" });
});

// Créer un article
router.post("/", (req, res) => {
  const articles = JSON.parse(fs.readFileSync(filePath));
  const { title, content } = req.body;
  const newArticle = {
    id: Date.now(),
    title,
    content
  };
  articles.push(newArticle);
  fs.writeFileSync(filePath, JSON.stringify(articles, null, 2));
  res.status(201).json(newArticle);
});

// Modifier un article
router.put("/:id", (req, res) => {
  let articles = JSON.parse(fs.readFileSync(filePath));
  const index = articles.findIndex(a => a.id == req.params.id);
  if (index === -1) return res.status(404).json({ error: "Article non trouvé" });
  articles[index] = { ...articles[index], ...req.body };
  fs.writeFileSync(filePath, JSON.stringify(articles, null, 2));
  res.json(articles[index]);
});
```

```
});  
  
// Supprimer un article  
router.delete("/:id", (req, res) => {  
  let articles = JSON.parse(fs.readFileSync(filePath));  
  const newArticles = articles.filter(a => a.id !== req.params.id);  
  fs.writeFileSync(filePath, JSON.stringify(newArticles, null, 2));  
  res.json({ message: "Article supprimé" });  
});  
  
module.exports = router;
```

## Étape 5 : Tester dans Postman ou le navigateur

### Exemples :

- GET `http://localhost:3000/api/articles` → liste des articles
- POST `http://localhost:3000/api/articles` + JSON :

```
{  
  "title": "Article Test",  
  "content": "Ceci est un test"  
}
```

## Résultat final

Un mini serveur API complet avec :

- CRUD d'articles
- données persistées en fichier `.json`
- requêtes testables depuis Postman

## Prolongement possible (si les étudiants sont prêts) :

- Ajouter une route `/api/users`
- Ajouter un champ `author` à chaque article
- Ajouter une validation simple (ex: pas de champ vide)
- Remplacer les fichiers JSON par MongoDB ou SQLite





# Comment tester l'API

---

Tu peux tester ton API de **3 façons différentes** :

1. Dans **Postman** (recommandé pour les débutants)
2. Avec **le navigateur** (pour les routes GET)
3. Avec **curl** (pour les plus à l'aise en ligne de commande)



## 1. Tester avec Postman (méthode visuelle recommandée)

### ♦ A. Lister tous les articles

- **Méthode** : GET
- **URL** : `http://localhost:3000/api/articles`
- **Action** : Cliquer sur "Send"
-  Tu dois voir un tableau JSON avec les articles

### ♦ B. Créer un nouvel article

- **Méthode** : POST
- **URL** : `http://localhost:3000/api/articles`
- **Onglet** : Body > raw > JSON

```
{  
  "title": "Nouveau test",  
  "content": "Contenu de l'article ajouté via Postman"  
}
```

-  Tu dois recevoir une réponse avec le nouvel article

### ♦ C. Voir un article par ID

- **Méthode** : GET
- **URL** : `http://localhost:3000/api/articles/1`

### ♦ D. Modifier un article

- **Méthode** : PUT
- **URL** : `http://localhost:3000/api/articles/1`
- **Body** : JSON

```
{  
  "title": "Titre modifié"  
}
```

#### ♦ E. Supprimer un article

- **Méthode** : DELETE
- **URL** : `http://localhost:3000/api/articles/1`

## 2. Tester via le navigateur (uniquement GET)

Dans la barre d'adresse du navigateur, entre :

`http://localhost:3000/api/articles`

Tu verras une réponse JSON avec les articles.

## 3. Tester avec `curl` (en terminal)

### Lister les articles

```
curl http://localhost:3000/api/articles
```

### Créer un article

```
curl -X POST http://localhost:3000/api/articles \  
-H "Content-Type: application/json" \  
-d '{"title":"Test Curl","content":"Contenu via terminal"}'
```

### Supprimer un article

```
curl -X DELETE http://localhost:3000/api/articles/1
```

## Astuce pour les débutants

### 👉 Toujours bien vérifier :

- Que ton **serveur Express est bien lancé** (`node server.js`)
- Que tu testes les **bonnes URL**
- Que tu **envoies le bon format JSON** dans le body

Parfait, finir avec un aperçu des **API publiques externes** (comme la météo, les devises ou les cartes) 🧑🌐

---

## 🌐 Bonus – Utiliser des API publiques (ex. météo, devises...)

En plus de créer leurs propres APIs, les développeurs peuvent **consommer des APIs déjà existantes**, fournies par des services tiers : météo, géolocalisation, finance, etc.

Ces APIs sont très utiles pour enrichir un site ou une application avec des **données en temps réel**.

---

### ☁️ Exemple 1 – API de météo (OpenWeatherMap)

- URL de base : `https://api.openweathermap.org/data/2.5/weather`
- Nécessite une **clé API** gratuite (après inscription)

#### ♦ Exemple d'appel

```
GET https://api.openweathermap.org/data/2.5/weather?
q=Paris&appid=VOTRE_CLE_API&units=metric
```

#### ♦ Réponse JSON (extrait)

```
{
  "name": "Paris",
  "main": {
    "temp": 24.5
  },
  "weather": [
    {
      "description": "peu nuageux"
    }
  ]
}
```

#### ♦ En JavaScript (fetch)

```
fetch("https://api.openweathermap.org/data/2.5/weather?
q=Paris&appid=VOTRE_CLE_API&units=metric")
  .then(res => res.json())
  .then(data => console.log(data));
```

## Exemple 2 – API de taux de change (ExchangeRate API)

- URL : <https://api.exchangerate-api.com/v4/latest/EUR>

Résultat :

```
{
  "base": "EUR",
  "rates": {
    "USD": 1.08,
    "GBP": 0.85
  }
}
```

## Exemple 3 – API de géolocalisation (ipinfo, ip-api)

- URL : <http://ip-api.com/json/>
- Renvoie l'adresse IP, la ville, le pays de l'utilisateur

## Ce que l'étudiant doit retenir

- Une API publique a :
  - Une **URL d'accès**
  - Parfois une **clé API** (gratuite ou payante)
  - Une documentation en ligne
- On consomme une API publique comme une API locale :
  - Avec `fetch`, `axios`, Postman, etc.
- On peut les utiliser dans leurs projets :
  - Afficher la météo sur un site perso
  - Convertir des devises
  - Localiser un utilisateur
  - Ajouter des cartes, des horaires, des infos utiles

## Quelques APIs populaires pour s'exercer

| API                 | Lien                                                                                                |
|---------------------|-----------------------------------------------------------------------------------------------------|
| OpenWeatherMap      | <a href="https://openweathermap.org/api">https://openweathermap.org/api</a>                         |
| ExchangeRate        | <a href="https://www.exchangerate-api.com/">https://www.exchangerate-api.com/</a>                   |
| IP Geolocation      | <a href="https://ip-api.com/">https://ip-api.com/</a>                                               |
| Joke API            | <a href="https://icanhazdadjoke.com/api">https://icanhazdadjoke.com/api</a>                         |
| PokeAPI             | <a href="https://pokeapi.co/">https://pokeapi.co/</a>                                               |
| Public APIs (liste) | <a href="https://github.com/public-apis/public-apis">https://github.com/public-apis/public-apis</a> |

---